Factory Lift Call System: Proposed Architecture & Effort Estimate

Document	Architecture proposal and effort estimate (detailed version)
Based on	*Factory Lift Call System Proposal* (1st & 2nd edition) and the screen mockups
Status	Draft for review
Date	2026-10-28

1. Summary

Workers at five production machines need a simple way to call the lift operator for **Product Pickup**, **Return Material Pickup** or **Trouble**. The operator needs to see the calls in order and respond. Both sides need to see whether the lift is running.

The original proposal used **two native WPF desktop apps** on the tablets. This document recommends a simpler setup: **one web application** served from a small on-site server, with the tablets running **Microsoft Edge in Windows kiosk mode**. It stays entirely on the customer's chosen stack (**.NET, SQL Server and IIS**) and uses the customer's existing SQL Server licence. It changes three things:

- **Nothing to install or update on the tablets.** A new version is deployed once, on the server.
- **Fewer moving parts.** It's one codebase and one deployable, and the screens are already designed as web pages.
- **Not tied to Windows tablets.** If Android tablets or a wall monitor are used later, they just open the same URL.

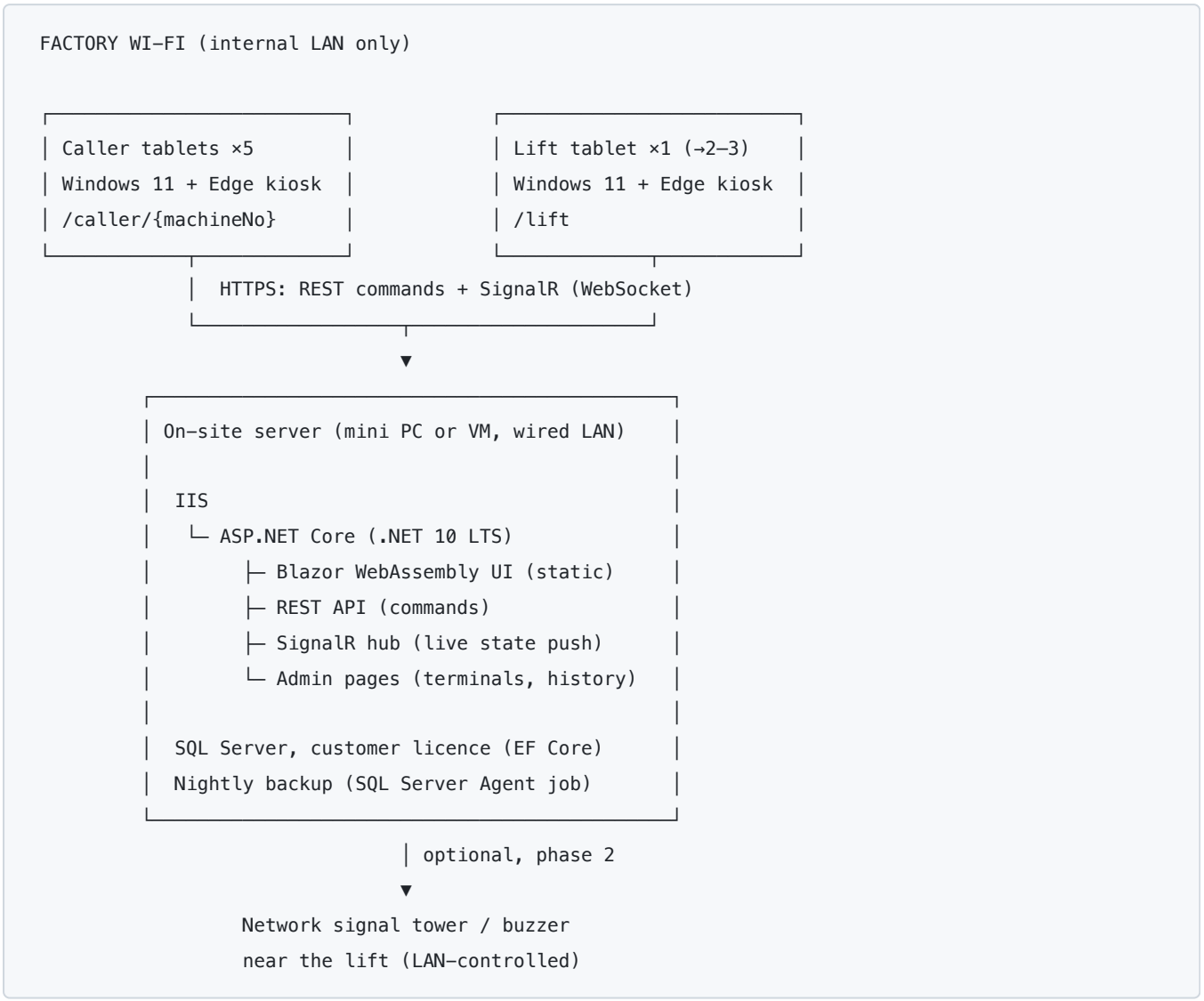
Rough effort: about **59 developer-days** and **24 tester-days** (likely case). One developer plus a part-time tester gets to go-live in about **12 weeks**, including a 2-week pilot on the factory floor.

2. Requirements Recap

Area	Requirement
Callers	5 tablets (Machines 1–5), touch-only, with 3 call buttons and a Cancel button
Caller display	Whether the call is being responded to, and whether the lift is running or stopped
Lift side	1 tablet now, 2–3 later. Shows which machine called, the reason, and the call order
Lift side actions	Respond to a call, and switch the lift between running and stopped
Network	All devices on the same factory Wi-Fi. Internal LAN only
Integration	No PLC integration. The operator sets the lift status by hand
Preferred stack	**.NET / C#** throughout, **SQL Server** (the customer already holds a licence), IIS. WPF was the original client choice

3. Recommended Architecture

3.1 Overview



3.2 How a call flows

1. A worker at Machine 3 taps **Product Pickup**. The tablet sends `POST /api/calls` with a request ID it generates itself, so a retry can never create a duplicate call.
2. The server saves the call in a single transaction and pushes the new **state snapshot** (all active calls plus the lift status) to every connected screen over SignalR.
3. The lift tablet plays a sound, shows the call at the end of the queue, and repeats the alert until someone responds.
4. The operator taps **Respond**. The server accepts it only if the call is still `Calling`, so with 2–3 lift terminals only the first tap wins. The new snapshot turns Machine 3's screen green with "Responding".
5. After the pickup the operator taps **Complete**, and the call moves to history.

3.3 Key design decisions

Decision	Choice	Why
Client type	**Web app in Edge kiosk mode** instead of WPF	Updates happen on the server only. Windows has a built-in kiosk mode (Assigned Access), and the browser restarts itself if it crashes. It also works on other devices later
UI framework	**Blazor WebAssembly** (C#)	The team stays in C# end to end. The UI runs in the browser, so a Wi-Fi drop doesn't lose it and it can show its own error state. Other .NET client options are compared in §5.3
Real-time updates	**SignalR (WebSockets)**, sending the **full snapshot** on every change, with **polling as an automatic fallback**	The state is tiny (at most a few dozen calls), so sending everything is simpler and more reliable than sending only the changes. After a reconnect, the screen is correct as soon as the next snapshot arrives. Other options (webhooks, message queues, MQTT) are compared in §5.2
Commands	**REST with request IDs** that make retries safe (idempotency)	A retry after a Wi-Fi blip never creates a second call. REST is also easy to test and log
Concurrency	Conditional update (`WHERE Status = 'Calling'`) plus a row version	With several lift terminals, two people can't both respond to the same call
Database	**SQL Server** under the customer's existing licence, accessed through EF Core	No extra licence cost, and the customer's IT team already knows how to run and back it up. It can be an existing instance on the network or installed on the app server, depending on the licence terms. SQL Server Agent handles backups and maintenance. Expected volume is only a few hundred rows a day
Hosting	**IIS**, ASP.NET Core in-process	It's what the customer asked for. WebSockets must be enabled and the app pool configured for long-lived connections (see §6)
Runtime	**.NET 10 LTS**	.NET 8, which the original proposal used, reaches end of support in **November 2026**. .NET 10 is supported until November 2028
Terminal identity	Each tablet has its own kiosk start URL with a **terminal token**, for example `/caller/3?t=…`	No login screen on the factory floor, but every request can be traced to a specific tablet. Tokens can be revoked on the admin page
Offline detection	**Presence comes from the SignalR connection itself**, plus a client heartbeat	The lift screen shows "Machine 4 offline" without a separate monitoring system

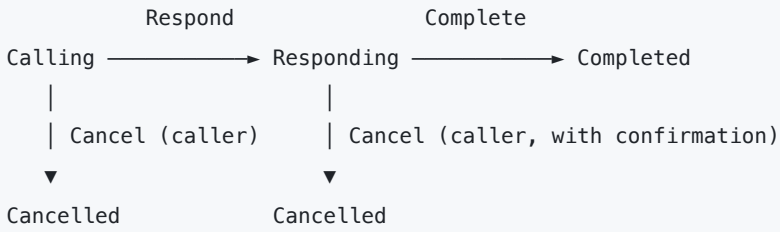
3.4 Tech stack

Layer	Technology
Tablet OS / shell	Windows 11 Pro, Assigned Access running **Microsoft Edge kiosk mode**
Front end	Blazor WebAssembly (.NET 10), CSS based on the existing mockups, Japanese and English resource files
Real-time	ASP.NET Core SignalR (WebSockets, with automatic fallback)
Back end	ASP.NET Core 10 Web API and SignalR hub, hosted in IIS
Data	SQL Server (customer licence, 2019 or later), EF Core 10 with migrations
Logging	Serilog writing rolling files plus a database audit table
Health	ASP.NET Core health checks (`/health`)

Layer	Technology
Tests	xUnit, `WebApplicationFactory` integration tests, bUnit for UI components, and a small simulator of fake terminals for soak testing
CI/CD	GitHub Actions or Azure DevOps: build, test, publish a zip. Deployment is a scripted copy to IIS (no internet access needed on the server)
Backup	SQL Server Agent job for nightly full backups to a network share, or the customer's existing SQL backup routine

4. Functional Design

4.1 Call lifecycle



4.2 Screens

Screen	Contents
Caller (`/caller/{n}`)	Machine number, lift status (Running/Stopped), response status (Idle, Calling, Responding), 3 large call buttons, Cancel Call (enabled only while a call is active), and a full-screen "Connection error" overlay when offline
Lift (`/lift`)	Running/Stopped toggle (long-press plus confirmation), call queue (order number, machine, reason with colour, icon and text, elapsed time, Respond button), "In progress" section with a Complete button, terminal online/offline strip. Sound alert on a new call, repeated every N seconds until someone responds, and cards blink after 3 minutes
Admin (`/admin` , office PC only)	Terminal list (online/offline, last seen, token reset), call history with filters and CSV export, settings (re-alert interval, overdue threshold, Trouble priority on/off)

4.3 Data model (minimal)

Table	Key columns
`Terminal`	Id, Name, Role (Caller or Lift), MachineNo, TokenHash, LastSeenAt, IsActive
`Call`	Id, MachineNo, CallType, Status, CreatedAt, RespondedAt, RespondedByTerminalId, CompletedAt, CancelledAt, RequestId (unique), RowVersion
`LiftState`	One row: IsRunning, UpdatedAt, UpdatedByTerminalId
`Setting`	Key, Value
`AuditEvent`	Id, At, TerminalId, Action, CallId, Details

4.4 API surface

Method	Endpoint	Purpose
`POST`	`/api/calls`	Create a call (with a request ID for safe retries)
`POST`	`/api/calls/{id}/respond`	Respond (lift side)
`POST`	`/api/calls/{id}/complete`	Complete (lift side)
`POST`	`/api/calls/{id}/cancel`	Cancel (calling machine only)
`PUT`	`/api/lift/state`	Set running or stopped
`GET`	`/api/state`	Full snapshot, used on startup and reconnect
`GET`	`/api/history`	History with filters and CSV export (admin)
Hub	`/hubs/state` → `StateChanged(snapshot)`	Live push to every screen

4.5 Recommended defaults for the open questions

These are suggested defaults to unblock development. Each needs the customer's confirmation, and most of them can be changed in settings.

Open question	Suggested default
Should Trouble calls jump the queue?	**Yes**, shown at the top in red. A setting can turn this off
Is a "Complete" step needed?	**Yes**, because it gives response-time and handling-time data. Calls still open after a configurable timeout are completed automatically
How many active calls per machine?	**One per call type.** A machine can request a product pickup and report trouble at the same time
Calls nobody answers	Repeat the sound every 30 s, cards blink after 3 min, and the event is logged. Escalation (signal tower or supervisor) is a phase 2 option
Calls while the lift is stopped	**Allowed.** The caller sees a red "Lift stopped" header, so they know the wait may be long
Who switches running/stopped?	The lift operator, on the lift tablet
Reports	A CSV export of the history is included. A KPI dashboard is a phase 2 option

5. Alternatives Considered

5.1 Changes from the original proposal

Topic	Original proposal	This proposal	Reason
Client	2 WPF apps (or 1 app with a startup switch)	1 web app in Edge kiosk mode	No client installs or updates; built-in kiosk and crash recovery
Runtime	.NET 8 LTS	.NET 10 LTS	.NET 8 support ends November 2026
Database	SQL Server Express suggested	SQL Server under the customer's existing licence	Already paid for. Includes SQL Server Agent for backups and maintenance
Live updates	SignalR events, with a polling fallback	SignalR full snapshot; `GET /api/state` on reconnect	Simpler and self-correcting

Topic	Original proposal	This proposal	Reason
Duplicate protection	Not specified	Request IDs on every command	Wi-Fi retries can't create duplicate calls
Admin	Not covered	Basic admin page (terminals, history, CSV export, settings)	Needed to run the system day to day
Alerting	Sound and screen	Sound with repeats, plus an optional signal tower	An operator driving the lift can't watch a tablet

If WPF is a hard requirement, the same server design still applies. Only the clients change. The extra effort is in §8.4.

5.2 How updates reach the tablets

The system has 6–8 tablets on one LAN and a few hundred calls a day. A new call only needs to appear within a second or two. Cost differences are relative to the estimate in §8.

Option	Fit for this system	Dev cost	Maintenance	Verdict
SignalR	Push to browsers with automatic reconnect, a C# client for Blazor, and online/offline status from the connection itself	Baseline	Low. IIS WebSocket and app pool settings are done once	**Recommended**
Polling (`GET /api/state` every 1–2 s)	8 clients × 1 request/s is negligible for the server. Each poll doubles as a heartbeat	–1 to –2 days	Lowest. No connection state, so IIS recycles and Wi-Fi drops don't matter	**Kept as the built-in fallback**
Server-Sent Events	One-way server push over plain HTTP, supported directly in .NET 10. Fits the "REST commands + pushed state" design	±0 to +1 day	Low	Viable, but no real advantage over SignalR for a C# team
Raw WebSockets	Reconnect, heartbeat and message handling must be written by hand	+3 to +4 days	Medium	Not worth it. SignalR is a well-tested wrapper around WebSockets
Webhooks	Server-to-server HTTP callbacks. Browsers and tablets can't receive incoming HTTP calls	n/a	n/a	**Not applicable** for screen updates. Possible later for outbound alerts (Teams, e-mail)
Message queue (RabbitMQ, Azure Service Bus, Kafka)	Built to decouple many backend services and buffer high volumes. Browsers can't connect directly, so SignalR would still be needed in front	+6 to +10 days	High. Another server to install, secure, monitor and back up	**Overkill** for one server and 8 clients
MQTT (e.g. Mosquitto broker, MQTTnet library for .NET)	Lightweight publish/subscribe widely used in factories (IIoT). Browsers connect over WebSockets. Physical buttons, signal towers and PLC gateways often support it natively	+5 to +8 days	Medium. Broker, access rules and TLS to maintain	**Future option** if many hardware devices are added

The hard parts of this system are business rules: call order, only one Respond per call, and no duplicate calls. They are enforced by a **database transaction** on the server. A queue or broker moves messages but doesn't enforce any of these rules, so it would add infrastructure without removing that work.

Because the server always sends the **full state**, the push channel only has to say "something changed". Switching to or falling back on polling is just calling `GET /api/state` on a timer. The client does this automatically when SignalR can't connect, and the cost is included in §8.1 item 9.

5.3 .NET client options

The customer wants to stay on .NET, so only .NET client technologies are compared. The server side (ASP.NET Core, SignalR, SQL Server on IIS) is the same for all three.

Option	Dev cost	Maintenance	When it makes sense
Blazor WebAssembly (recommended)	Baseline	Low. Updates are deployed on the server only	Default. The UI keeps running in the browser through Wi-Fi drops and shows its own error state
Blazor Server (UI runs on the server)	–5 to –8 days. No separate API layer, and live updates are built in	Low	Only if the Wi-Fi survey shows reliable coverage. The UI depends on a constant connection: a drop freezes the screen with "Reconnecting…", and longer drops lose the session. .NET 10 improves this, but it remains a risk on factory Wi-Fi
WPF (native Windows app, as in the original proposal)	+8 to +14 days (see §8.4)	Higher. An installer and auto-update are needed on every tablet	If a native app is required, for example to use local hardware or to match existing WPF applications

5.4 Conclusion

1. **Blazor WebAssembly** on ASP.NET Core, with the customer's licensed **SQL Server**, hosted on IIS.
2. **Keep SignalR** for live updates, with polling as the automatic fallback.
3. **Don't use webhooks or a message queue** in the core system. Webhooks can be a small phase 2 add-on for Teams or e-mail alerts.
4. **Keep MQTT in mind** if physical call buttons, signal towers or PLC data are added later.
5. **Blazor Server** can save about a week of development, but only if the Wi-Fi survey shows reliable coverage.

6. Deployment & Operations

6.1 Hardware

Item	Qty	Notes
Server	1	Small industrial mini PC or a VM on an existing server. Windows Server or Windows 11 Pro, **wired LAN**, static IP or DNS name, on a **UPS**. The database can use an existing SQL Server instance on the customer's network instead of a local install
Caller tablets	5 (+1 spare)	Windows 11 Pro (Home has no Assigned Access). Glove-friendly touch screen. Wall or arm mount with permanent power
Lift tablet	1 (+1 spare)	As above, plus an **external powered speaker**, because tablet speakers are too quiet in a factory
Wi-Fi	—	A **site survey** at every tablet location and along the lift route. One SSID for these devices
Signal tower (optional)	1	A LAN-controlled light tower with buzzer near the lift (phase 2)

6.2 Server configuration checklist

- Install the IIS **WebSocket Protocol** feature. Without it, SignalR falls back to slower transports.
- App pool settings: Start Mode **AlwaysRunning**, Idle Time-out **0**, periodic recycling turned off or moved to a scheduled night-time recycle.
- Enable preload on the site so the app starts with IIS.
- SQL Server: nightly full backup through a SQL Server Agent job (or the customer's existing backup routine), copied to a network share and kept for 14 days. Call history is kept for 1–3 years (purge job).
- Use an HTTPS certificate from the internal CA, if IT can provide one. Plain HTTP is acceptable on an isolated LAN, but HTTPS is recommended.

6.3 Tablet (kiosk) configuration checklist

- **Assigned Access** single-app kiosk running Microsoft Edge, with the start URL set per tablet. Edge reopens the page automatically if it is closed or crashes.
- Edge policies:
- **Autoplay allowed** for the app URL, so alert sounds play without a tap first.
- Translation pop-ups and the first-run experience turned off.
- Power settings: the screen never turns off and the device never sleeps. Wi-Fi adapter power saving is turned off.
- Use the vendor's **battery charge limit** (for example 80%) to avoid battery swelling on always-plugged tablets.
- Windows Update active hours set outside production time.
- A **provisioning guide** so a spare tablet can be set up in under 30 minutes.

6.4 Run-time behaviour on failures

Failure	What users see	Recovery
A tablet loses Wi-Fi	Full-screen "Connection error". Buttons are disabled so nobody thinks a call was sent	Reconnects and resyncs automatically
Server or IIS restart	All screens show "Connection error" for a few seconds	Automatic reconnect. Active calls are restored from the DB
Server down for a long time	"Connection error" on every screen	Use the agreed fallback (phone or radio). Restore runbook plus a spare server or VM image
Browser crash	—	Kiosk mode restarts Edge automatically

7. Delivery Plan

Agile delivery in 2-week sprints, with a review on the factory floor at the end of each sprint.

Week	Activities
1–2	Requirements workshop, decisions on the open questions, clickable mockups reviewed on site, technical design
3–4	Sprint 1: project setup, CI, data model, call state machine, REST API, SignalR snapshot, caller screen
5–6	Sprint 2: lift screen (queue, timers, sound alerts, Respond and Complete, running/stopped toggle), offline detection
7	Sprint 3 (short): admin pages, history and CSV export, settings, Japanese and English UI text
8	Server and kiosk setup scripts, system test cycle 1

Week	Activities
9	On-site testing on real tablets and Wi-Fi, fixes, regression cycle 2
10–11	UAT and **pilot on the floor** (live use with a fallback procedure), adjustments
12	Go-live, handover of operation and admin guides

The tester starts at about 50% from week 3 (test planning) and works close to full time in weeks 8–11.

8. Effort Estimate

Units are **person-days**. Ranges are Low / Likely / High. The estimate assumes a mid-to-senior C# developer with ASP.NET Core experience.

8.1 Developer

#	Work item	Low	Likely	High
	Discovery & design			
1	Requirements workshop and decisions on the open questions	1.5	2	3
2	Clickable UI mockups and on-site review (2 rounds)	2	3	5
3	Technical design (data model, API, deployment)	1.5	2	3
	Back end			
4	Solution setup, CI pipeline, EF Core migrations	1.5	2	3
5	Call state machine, REST commands, validation, duplicate protection	3	4	6
6	SignalR hub, snapshot push, presence and offline detection	2	3	4
7	Terminal registration and tokens, settings	1	1.5	2
8	Logging, health check, audit trail	1	1	2
	Front end			
9	Client shell: SignalR client, reconnect and resync, polling fallback, connection overlay, Japanese and English text	2.5	3	4
10	Caller screen	3	4	5
11	Lift screen (queue, timers, Respond/Complete, long-press toggle, sound and repeat alerts)	4.5	6	8
12	Admin pages: terminals, history, CSV export, settings	2	3	4
	Deployment			
13	Server setup: IIS, database creation, SQL Agent backup job, install scripts	1.5	2	3
14	Kiosk setup: Assigned Access, Edge policies, power and Wi-Fi settings, provisioning guide	1.5	2	3
	Quality & handover			
15	Automated tests (unit, integration, concurrency)	2	3	4
16	Soak-test simulator (fake terminals)	1	1	2
17	Bug fixing (about 15% of build effort)	4	6	9
18	On-site test support	1.5	2	3
19	UAT and pilot support (2 weeks of support after release)	4	6	8

#	Work item	Low	Likely	High
20	Operation and admin guides, handover	1.5	2	3
	Developer total	**42.5**	**58.5**	**84**

8.2 Tester

#	Work item	Low	Likely	High
1	Requirements and mockup review, acceptance criteria	1	1.5	2
2	Test plan and test cases	2	3	4
3	Test environment (tablets, test server, simulated terminals)	1	1.5	2
4	Functional testing, cycle 1 (caller, lift, admin)	3	4	5
5	Resilience and negative tests (Wi-Fi drop, server restart, sleep/wake, double tap, simultaneous Respond)	2	3	4
6	Tests with several lift terminals at once	1	1.5	2
7	On-site testing on real tablets, Wi-Fi coverage walk along the lift route	2	3	4
8	Regression, cycle 2	1.5	2	3
9	48-hour soak test monitoring	0.5	1	1.5
10	UAT support and pilot monitoring	2	3	4
	Tester total	**16**	**23.5**	**31.5**

8.3 Totals

Role	Low	Likely	High
Developer	42.5	58.5	84
Tester	16	23.5	31.5
Total	**58.5**	**82**	**115.5**

- **Calendar time:** about **12 weeks** with 1 developer and 1 tester (part-time early, near full-time later). With 2 developers, about **8–9 weeks**.
- **Not included:** project management and customer coordination (add 10–15%), hardware purchase and installation, the Wi-Fi site survey, travel, and Windows licences. The SQL Server licence is provided by the customer.

8.4 Variants and optional add-ons

Item	Developer	Tester	Notes
WPF clients instead of web (same server)	+8 – 14	+2 – 4	Two XAML UIs, MSIX or ClickOnce packaging and auto-update, Shell Launcher kiosk setup, install and upgrade testing
Network signal tower or buzzer near the lift	3 – 5	1 – 2	Lights and sounds on a new call. Very useful if the operator is away from the tablet
KPI dashboard (response times; calls by machine, type and hour)	5 – 8	2 – 3	Uses the existing history data

Item	Developer	Tester	Notes
PLC integration for automatic running/stopped status	5 – 10	2 – 4	Depends on the PLC model and protocol. Needs a technical spike first
Supervisor escalation (e-mail or Teams) for unanswered Trouble calls	2 – 3	1	Needs a mail server or internet access

8.5 AI-assisted delivery estimate

This estimate assumes the developer uses an AI coding assistant (for example Claude Code) for most of the code, automated tests, scripts and documentation. The developer still reviews all of it. Work tied to people, hardware or the factory floor changes little.

Developer (likely case)

#	Work item	Standard	AI-assisted	Notes
1	Requirements workshop	2	1.5	Meetings unchanged. AI prepares the agenda and writes up decisions
2	Clickable mockups and on-site review	3	1.5	Mockups are generated quickly. On-site review time stays
3	Technical design	2	1	Drafted by AI, reviewed by the developer
4–8	Back end (setup, API, SignalR, tokens, logging)	11.5	6	Standard ASP.NET Core patterns built from a clear spec
9–12	Front end (shell, caller, lift, admin)	16	7.5	Screen code is fast to write. Fine-tuning on real tablets (touch, sound, long-press) is not
13–14	Server and kiosk setup	4	3	Scripts are generated. Running and fixing them on real hardware is hands-on
15–16	Automated tests and simulator	4	1.5	One of the largest savings
17	Bug fixing	6	3.5	Less code to fix, but generated code needs careful review, especially the concurrency rules
18–19	On-site support, UAT and pilot	8	7	Mostly fixed by calendar time and site visits
20	Documentation and handover	2	1	Drafted from the code
	Developer total	**58.5**	**≈ 33**	Range 26–45

Tester (likely case)

Work item	Standard	AI-assisted	Notes
Requirements review, test plan, test cases	4.5	2.5	Test cases drafted from the spec
Test environment	1.5	1	The simulator is generated
Functional testing, cycle 1	4	3	Touch testing on the tablets stays manual
Resilience and multi-terminal tests	4.5	3.5	Physical tests (Wi-Fi loss, sleep/wake) stay manual. Simultaneous-tap tests are scripted

Work item	Standard	AI-assisted	Notes
Regression, cycle 2	2	1	Mostly covered by an automated end-to-end browser suite (Playwright)
On-site, soak, UAT and pilot	7	7	Fixed by calendar time and site visits
Tester total	**23.5**	**≈ 18**	Range 14–24

Summary

Role	Standard (likely)	AI-assisted (likely)	AI-assisted range
Developer	58.5	33	26 – 45
Tester	23.5	18	14 – 24
Total	**82**	**51**	**40 – 69**

- **Calendar time:** about **8–9 weeks** instead of 12. After that, the limits are customer decisions, on-site reviews and the 2-week pilot, not coding.
- **Quality:** thorough automated tests, including an end-to-end browser suite, are cheap with AI. Regression coverage ends up higher than the standard estimate allows for.
- **Conditions:** the developer is experienced in .NET and reviews all generated code. With a less experienced developer the savings shrink and the risk of subtle defects (concurrency, security) increases.
- **Confidence:** the AI-assisted figures are judgement-based. Published productivity results vary widely, and this project (new, small, clear spec, mainstream stack) is a favourable case. Measure actual progress after the first sprint and re-plan if needed. Keep contingency: factory Wi-Fi and real hardware are where projects like this usually overrun.

9. Risks & Mitigations

Risk	Impact	Mitigation
Weak Wi-Fi at some machine locations or along the lift route	Missed or late calls	Site survey before go-live. Connection-error overlay so failures are visible. Coverage walk during on-site testing
The operator can't see or hear the tablet while driving the lift	Calls wait too long	External speaker, repeating alerts, optional signal tower (§8.4)
Browser blocks the alert sound	Silent alerts	Edge autoplay policy for the app URL, checked during kiosk setup
IIS recycles the app or drops WebSockets	Brief disconnects	App pool settings in §6.2. Clients reconnect and resync automatically
The single server fails	System down	UPS, nightly backups, restore runbook, spare VM image, agreed phone or radio fallback
Always-plugged tablets: battery swelling, sleep, Windows updates	Device downtime	Charge limit, power policy, update active hours, spare tablets
Open business rules not decided in time	Rework	Defaults in §4.5, made configurable. Decisions fixed in weeks 1–2
Hard requirement to stay on WPF	Extra effort	Server design unchanged. Estimate in §8.4

10. Assumptions

- The customer provides the tablets, the server or VM, the Wi-Fi network, network access, and a contact on the factory floor for reviews and UAT.
- 5 caller terminals and up to 3 lift terminals, at one site, for one lift.
- The UI is in Japanese, with English text available. Documentation is in English (Japanese translation not included).
- No external system integration (ERP, MES, PLC) in the base scope.
- The development team can reach the site for on-site testing (weeks 9–11).